Projeto de Implementação de um Compilador para a Linguagem TPP

Análise Semântica (Trabalho -- 3a. parte)

Prof. Rogério Aparecido Gonçalves

> **Resumo:** *Este documento apresenta a especificação da 2a. parte do trabalho de implementação da disciplina. O objetivo nessa etapa é projetar e implementar a fase de* ***Análise Semântica*** *do compilador para a linguagem* ***TPP****.*

Análise Semântica

A **Análise Semântica** consiste em uma *"segunda passada"* do compilador no código, pois o código fonte é novamente analisado desde o início na **AST**, porém agora para realizar a anotação de atributos, inferência de tipos e verificações se as construções sintáticas fazem algum sentido para a linguagem.

Como entrada para a *Análise Semântica* temos a Árvore Sintática Abstrata (AST) produzida pela fase anterior, a *Análise Sintática* com a utilização da lista de *tokens* identificados pela *Análises Léxica* na construção da árvore sintática pela derivação das regras sintáticas da linguagem.

O principal motivo da Análise Semântica é verificar se existem erros de contexto para a linguagem. Um conjunto de Regras Semânticas devem ser verificadas.

Regras Semânticas

As **Regras Semânticas** que precisam ser verificadas para a identicação de erros semânticos ou de contexto são:

> **Gramática TPP v1.1:** a versão 1.1 da gramática (docs/grammar/grammar.md) acrescentou atribuição múltipla e encadeada, alvos entre parênteses (tuplas), retorno múltiplo, inicialização de arranjos e declaração com inicialização. O analisador sintático aceita algumas construções que só podem ser validadas na Análise Semântica; elas estão descritas nos itens 4.3 a 7. A estrutura da árvore produzida para essas construções está na seção Árvore Sintática das extensões da TPP v1.1.

1. **Funções e Procedimentos:** Verificações relacionadas a funções e procedimentos.

   1.1. **Função Principal.** Todo programa escrito em **TPP** deve ter uma função **principal** declarada que inicializa a execução do código. Verificar a existência de uma função com o nome de *"principal"*, se não existir deve apresentar a mensagem:

   Tipo: Erro.

   Chave: `ERR-SEM-MAIN-NOT-DECL`.

   Mensagem: `Função 'principal' não declarada.`

   1.2. **Declaração de Função:** Funções precisam ser declaradas antes de serem chamadas. Caso contrário a mensagem de erro deve ser emitida:

   Tipo: Erro.

   Chave: `ERR-SEM-CALL-FUNC-NOT-DECL`.

   Mensagem: `Chamada à função 'func' que não foi declarada.`

   1.3. **Identificador de Função:** Uma função ou procedimento é identificado

pelo seu tipo *retorno*, seu *nome* e seus *parâmetros formais*.

A quantidade de *parâmetros reais* de uma chamada de função/procedimento `func` deve ser igual a quantidade de *parâmetros formais* da sua definição. Caso contrário, gerar as mensagens:

Tipo: Erro.

Chave: `ERR-SEM-CALL-FUNC-WITH-FEW-ARGS` ou `ERR-SEM-CALL-FUNC-WITH-MANY-ARGS`

Mensagem: `Chamada à função 'func' com número de parâmetros menor que o declarado.`

Mensagem: `Chamada à função 'func' com número de parâmetros maior que o declarado.`

```
1.4. __Declarada e não utilizada:__ Uma função pode ser declarada e não utilizada.
Se isto acontecer uma aviso deverá ser emitido:

__Tipo:__ Aviso.

__Chave:__ `WAR-SEM-FUNC-DECL-NOT-USED`.

__Mensagem:__ `Função 'func' declarada, mas não utilizada.`.
```

```
1.5. __Retorno de Função:__ Uma função deve retornar um valor de tipo compatível
com o tipo de retorno declarado.

A função principal normalmente é do tipo `inteiro`, assim é esperado que seu
retorno seja um valor `inteiro`, se não apresenta um `retorna(0)`, a mensagem deve
ser gerada:

__Tipo:__ Erro.

__Chave:__ `ERR-SEM-FUNC-RET-TYPE-ERROR`.

__Mensagem:__ `Função 'principal' deveria retornar 'inteiro', mas retorna 'vazio'`.

Para as outros funções o tipo de retorno deve ser compatível com o declarado.

Uma função qualquer não pode fazer uma chamada à função principal. Devemos
verificar ser existem alguma chamada para a função principal partindo de qualquer
outra função do programa.

__Tipo:__ Erro.
```

```
__Chave:__ `ERR-SEM-CALL-FUNC-MAIN-NOT-ALLOWED`.

__Mensagem:__ `Chamada à função 'principal' não permitida.`

Se a função principal fizer uma chamada para ela mesmo, a mensagem de aviso deve
ser emitida:

__Tipo:__ Aviso.

__Chave:__ `WAR-SEM-CALL-REC-FUNC-MAIN`.

__Mensagem:__ `Chamada recursiva para 'principal'.`
```

2. **Variáveis.**

   2.1. O **identificador de variáveis locais e globais**: nome, tipo e escopo devem ser aramazenados na Tabela de Símbolos.
   Variáveis devem ser **declaradas** e **inicializadas**, antes de serem **utilizadas** (leitura).
   Lembrando que uma variável pode ser declarada:

   - no escopo do procedimento (como expressão ou como parâmetro formal).
   - no escopo global.

   Se houver a tentativa de leitura ou escrita de qualquer variável não declarada a seguinte mensagem:

   Tipo: Erro.

   Chave: `ERR-SEM-VAR-NOT-DECL`

   Mensagem: `Variável 'a' não declarada.`

   Se uma variável 'a' for apenas declarada e não for inicializada (escrita). Se houver a tentativa de leitura de uma variável 'a' declarada, mas não inicializada:

   Tipo: Aviso.

   Chave: `WAR-SEM-VAR-DECL-NOT-INIT`

   Mensagem: `Variável 'a' declarada e não inicializada.`

```
Se uma variável 'a' for declarada e não for utilizada (não lida), o analisador deve
gerar a mensagem:

__Tipo:__ Aviso.

__Chave:__ `WAR-SEM-VAR-DECL-NOT-USED`

__Mensagem:__ `Variável 'a' declarada e não utilizada.`

*Warnings* deverão ser mostrados quando uma variável for declarada mais de uma vez.
Se uma variável 'a' for declarada duas vezes no mesmo escopo, o aviso deve ser
emitido:

__Tipo:__ Aviso.

__Chave:__ `WAR-SEM-VAR-DECL-PREV`

__Mensagem:__ `Variável 'a' declarada anteriormente com o tipo 'inteiro'.`
```

3. **Atribuição de tipos distintos e Coerções implícitas.** Na atribuição devem ser verificados se os tipos são compatíveis. *Warnings* deverão ser mostrados quando ocorrer uma coerção implícita de tipos (inteiro<->flutuante).

   Atribuição de variáveis ou de números ou de resultados de chamadas à funções ou resultados de expressões de tipos distintos devem gerar a mensagem:

   Tipo: Aviso.

   Chave: `WAR-SEM-ATR-DIFF-TYPES-IMP-COERC-OF-VAR`

   Mensagem: `Atribuição de tipos distintos. Coerção implícita do valor de 'x' do tipo 'inteiro' para 'y' que é 'flutuante'.`

```
__Tipo:__ Aviso.

__Chave:__ `WAR-SEM-ATR-DIFF-TYPES-IMP-COERC-OF-NUM`

__Mensagem:__ `Atribuição de tipos distintos. Coerção implícita do valor atribuído
'5.0' do tipo 'flutuante' para 'x' que é 'inteiro'.`

Por exemplo, uma variável `a` recebe uma expressão `b + c`. Os tipos declarados
para `a` e o tipo resultado da inferência do tipo da expressão `b + c` deverão ser
compatíveis. Se `b` for `inteiro` e `c` for `inteiro`, o tipo resultante da
expressão será também `inteiro`.
```

```
Se assumirmos, por exemplo, que `b` é do tipo `inteiro` e `c` do tipo `flutuante`, o resultado pode ser `flutuante` (se assumirmos isso para nossa linguagem). O que faria a atribuição `a := b + c` apresentar tipos diferentes e a mensagem deve ser apresentada:

__Tipo:__ Aviso.

__Chave:__ `WAR-SEM-ATR-DIFF-TYPES-IMP-COERC-OF-EXP`

__Mensagem:__ `Atribuição de tipos distintos. Coerção implícita do valor de 'expressão' do tipo 'flutuante' para 'a' que é 'inteiro'.`

O mesmo pode acontecer com a atribuição de um retorno de uma função, se os tipos forem incompatíveis o usuário deve ser avisado:

__Tipo:__ Aviso.

__Chave:__ `WAR-SEM-ATR-DIFF-TYPES-IMP-COERC-OF-RET-VAL`

__Mensagem:__ `Atribuição de tipos distintos. Coerção implícita do valor retornado por 'func' do tipo 'inteiro' para 'y' que é 'flutuante'.`

Se algum argumento de uma chamada de função for diferente do tipo declarado o aviso deve ser dado:

__Tipo:__ Aviso.

__Chave:__ `WAR-SEM-ATR-DIFF-TYPES-IMP-COERC-OF-FUNC-ARG`

__Mensagem:__ `Chamada à função 'func' com Coerção implícita do valor do argumento tipo 'flutuante' diferente do parâmetro declarado 'inteiro'`
```

4. **Arranjos.** Na linguagem `TPP` é possível declarar arranjos, pela sintaxe da linguagem o índice de um arranjo é inteiro e isso deve ser verificado. Na tabela de símbolos devemos armazenar se uma variável declarada tem um tipo, se é uma variável escalar ou um vetor ou uma matriz. Podemos armazenar um campo 'dimensões', que '0': escalar, '1': arranjo unidimensional (vetor) e '2': arranjo bidimensional (matriz) e assim por diante.

   4.1. Encontrado a referência a um arranjo, seu o índice, seja um número, variável ou expressão deve ser um `inteiro`. Isso vale tanto na declaração, em que a expressão entre colchetes é o tamanho da dimensão (`inteiro: v[n]`, `flutuante: M[2][n + 1]`), quanto no uso, em que ela é o índice do elemento (`v[i]`, `M[0][j - 1]`). Do contrário, a mensagem deve ser gerada:

Tipo: Erro.

Chave: `ERR-SEM-ARRAY-INDEX-NOT-INT`

Mensagem: `Indice de array 'x' não 'inteiro'.`

A árvore permite distinguir os dois usos de `v[...]`: na declaração, a `var` está dentro de `lista_variaveis` (tamanho); numa atribuição ou expressão, está dentro de um `fator` (índice).

4.2. Se o acesso ao elemento do arranjo estiver fora de sua definição, por exemplo um vetor `A` é declarado como tendo `10` elementos (0 a 9) e há um acesso ao `A[10]`, a mensagem de erro deve ser apresentada. A mesma mensagem deve ser apresentada na declaração quando o tamanho de uma dimensão não for maior que zero, como em `inteiro: v[0]` ou `inteiro: v[-1]`. As duas verificações valem para os valores conhecidos durante a compilação, como números:

Tipo: Erro.

Chave: `ERR-SEM-ARRAY-INDEX-OUT-OF-RANGE`

Mensagem: `Indice de array 'A' fora do intervalo (out of range).`

4.3. **Inicialização de arranjos.** Um inicializador (`{1, 2, 3}`, ou `{{1, 2}, {3, 4}}` para matrizes) pode aparecer na declaração (`inteiro: v[5] := {1, 2, 3, 4, 5}`), na inicialização global (`v := {1, 2}`) e em atribuições. O inicializador se ajusta à forma do alvo:

- **Inicialização parcial:** se o alvo tem mais posições que o inicializador, as restantes recebem `0`. `inteiro: v[5] := {1, 2}` equivale a `{1, 2, 0, 0, 0}`. Não gera mensagem.
- **Alvo escalar:** se o alvo é um elemento (`v[4]`) ou uma variável escalar, é usado o primeiro elemento: `v[4] := {1, 2}` equivale a `v[4] := 1`.
- **Linha de matriz:** com `inteiro: M[2][3]`, `M[0] := {1, 2}` preenche a linha 0 com `{1, 2, 0}`.
- **Elementos excedentes:** se o inicializador tem mais elementos que o

alvo comporta (inclusive no caso do alvo escalar), os excedentes são descartados e o aviso deve ser emitido:

Tipo: Aviso.

Chave: `WAR-SEM-ARRAY-INIT-EXCESS-ELEMS`

Mensagem: `Elementos excedentes no inicializador de 'v' foram descartados.`

Se o aninhamento do inicializador tiver mais níveis que as dimensões do alvo (por exemplo, `{{1, 2}}` para um vetor), a mensagem de erro deve ser emitida:

Tipo: Erro.

Chave: `ERR-SEM-ARRAY-INIT-DIM`

Mensagem: `Inicializador de 'v' tem mais dimensões que o arranjo.`

O inicializador é um valor completo e não pode ser usado como operando de uma expressão (`x := {1} + 2`) nem como argumento de `escreva`:

Tipo: Erro.

Chave: `ERR-SEM-ARRAY-INIT-IN-EXP`

Mensagem: `Inicializador de arranjo não pode ser usado em uma expressão.`

Os elementos do inicializador devem ser compatíveis com o tipo do arranjo; as coerções implícitas seguem as regras do item 3.

5. **Atribuição múltipla, encadeada e tuplas.**

5.1. **Alvos da atribuição.** Dentro das funções, o analisador sintático lê o lado esquerdo da atribuição como uma lista de expressões (como o Python), e por isso aceita `f() := 1`, `1 := x` e `a + b := c`. Cada alvo deve ser uma variável (`a`, `v[i]`, `M[i][j]`) ou uma tupla de variáveis (`(a, b)`). Caso contrário:

Tipo: Erro.

Chave: `ERR-SEM-ATR-INVALID-TARGET`

Mensagem: `Alvo inválido na atribuição: 'f()' não é uma variável.`

No escopo global (`inicializacao_variaveis`) e nos elos de `atribuicao_encadeada` da declaração, o analisador sintático já só aceita variáveis como alvo.

5.2. **Quantidade de alvos e de valores.** Em `a, b := c, d`, a quantidade de alvos deve ser igual à de valores. Antes de comparar, cada lado deve ser normalizado:

- se um lado tem um único elemento e ele é uma tupla, a tupla é desmontada nos seus elementos: `(a, b) := 1, 2` e `a, b := (1, 2)` têm dois alvos e dois valores;
- se o lado direito é uma única chamada de função, a quantidade de valores é a quantidade de tipos de retorno declarados: `a, b := funct(10, 3)` com `(inteiro, flutuante) funct(...)` tem dois valores.

Se as quantidades forem diferentes:

Tipo: Erro.

Chave: `ERR-SEM-ATR-DIFF-NUM-TARGETS-VALUES`

Mensagem: `Atribuição com 2 alvos e 3 valores.`

Cada par alvo/valor é verificado individualmente pelas regras de tipos e coerções do item 3. Todo o lado direito é avaliado antes de qualquer alvo receber valor, de modo que `a, b := b, a` troca os valores de `a` e `b`.

5.3. **Tuplas fora da atribuição.** Uma tupla (`(a, b)`) só é permitida como lado inteiro de uma atribuição, isto é, como único alvo ou como único valor. Em qualquer outro lugar (`escreva((1, 2))`, `x := (1, 2) + 3`, argumento de função, condição de `se`), a mensagem deve ser emitida:

Tipo: Erro.

Chave: `ERR-SEM-TUPLE-NOT-ALLOWED`

Mensagem: `Tupla não permitida neste contexto.`

5.4. **Atribuição encadeada.** Em `a := b := 0` (dentro de funções) o valor é avaliado uma única vez e as atribuições são feitas da direita para a esquerda, do elo mais interno para o mais externo: primeiro `b := 0`, depois `a` recebe o valor de `b`. Cada elo é verificado pelas regras dos itens 3, 5.1 e 5.2.

5.5. **Declaração com inicialização encadeada.** Em `inteiro: a := b := 1`, apenas `a` é declarada. A variável `b` deve ter sido declarada antes; caso contrário, aplica-se `ERR-SEM-VAR-NOT-DECL` (item 2.1). As duas recebem `1`, com as verificações de tipos do item 3. Na declaração simples (`inteiro: a, b := 1, 2`), a quantidade de variáveis declaradas e de valores segue a regra 5.2.

6. **Retorno múltiplo.** Uma função pode declarar vários tipos de retorno `(inteiro, flutuante) funct(inteiro: a, inteiro: b)` e retornar vários valores `retorna(q, r)`.

   6.1. A quantidade de valores de cada `retorna(...)` deve ser igual à quantidade de tipos declarados:

   Tipo: Erro.

   Chave: `ERR-SEM-FUNC-RET-NUM-ERROR`

   Mensagem: `Função 'funct' deveria retornar 2 valores, mas retorna 1.`

   O tipo de cada valor é comparado com o tipo declarado na mesma posição, pelas regras do item 1.5.

   6.2. Uma chamada a uma função com mais de um retorno só pode aparecer como único valor do lado direito de uma atribuição (`q, r := funct(10, 3)`). Usada em uma expressão (`x := funct(10, 3) + 1`), como argumento ou em `escreva`, a mensagem deve ser emitida:

   Tipo: Erro.

   Chave: `ERR-SEM-CALL-FUNC-MULTI-RET-IN-EXP`

   Mensagem: `Chamada à função 'funct', que retorna 2 valores, usada em uma expressão.`

7. **Expressões.**

7.1. **Comando sem efeito.** Uma expressão usada como comando só tem efeito se for uma chamada de função. Como os comandos terminam no fim da linha, `x := a` seguido de `+ b` na linha de baixo são dois comandos, e o segundo não faz nada. Também é o caso de `vet[i] = j`, uma comparação usada no lugar de `vet[i] := j`. Nesses casos, o aviso deve ser emitido:

Tipo: Aviso.

Chave: `WAR-SEM-EXP-STMT-NO-EFFECT`

Mensagem: `Expressão usada como comando não tem efeito.`

7.2. **Comparações encadeadas.** `a < b < c` é aceito pela sintaxe e equivale a `(a < b) < c`, que compara o resultado de uma comparação com `c`. O aviso deve ser emitido:

Tipo: Aviso.

Chave: `WAR-SEM-REL-CHAIN`

Mensagem: `Comparação encadeada: o resultado de uma comparação é comparado com outro valor.`

7.3. **Precedência de `&&` e `||`.** Na gramática, `&&` e `||` têm a mesma precedência e associam à esquerda: `a || b && c` é avaliado como `(a || b) && c`. A Análise Semântica e a Geração de Código devem respeitar a árvore produzida.

8. E outras situações descritas nos arquivos de testes.

Para a realização da análise dos erros apontados, a construção da **Tabela de Símbolos** deve ser realizada adequadamente. Uma Tabela de Símbolos pode começar a ser construída desde as análises anteriores. Por exemplo, na **Análise Léxica** temos os `TOKENS` e os *lexemas* que casaram com a expressão regular que gerou cada um desses tokens. Temos também o número da linha e da coluna que o lexema foi encontrado no código fonte de entrada. Desta forma, as entradas da **Tabela de Símbolos** podem ir sendo preenchidas com essas informações iniciais `<TOKEN, LEXEMA, LINHA, COLUNA>`.

Na **Análise Sintática** quando o reconhecimento entrar na função que trata a regra, por exemplo, na regra de `declaração de variáveis`, todas as variáveis da lista de identificadores irão assumir o tipo declarado:

`inteiro: a, b, c[10]`

Que entraria na regra:

```
declaracao_variaveis ::= tipo DOIS_PONTOS lista_variaveis
                         | tipo DOIS_PONTOS lista_variaveis ATRIBUICAO
lista_expressoes
                         | tipo DOIS_PONTOS lista_variaveis ATRIBUICAO
atribuicao_encadeada
```

A declaração indica que os elementos da `lista_variaveis`, que são as variáveis escalares `a` e `b` e o arranjo unidimensional `c` terão por declaração o `tipo` que no exemplo é `inteiro`.

Tabela de Símbolos

Considere o código do Exemplo:

```
inteiro: a, b[10], c[3][5]

b := 10

inteiro func(inteiro: x, inteiro: y)
  inteiro: res
  res = x + y;
  retorna(res)
fim

inteiro principal()
  leia(a)
  leia(b)

  b[0] := a
  b[1] := b
  c[0][1] := func(a,b)

  escreva(c[3])
  retorna(0)
```

```
fim
```

Uma ideia de *Tabela de Símbolos* pode ser vista na tabela:

Token	**Lexema**	**Tipo**	**dim**	**tam_dim1**	**tam_dim2**	**escopo**	**init**	**linha**	**coluna**
ID	"a"	int	0	1	0	global	N	1	9
ID	"b"	int	1	10	0	global	N	1	9
ID	"c"	int	2	3	5	global	N	1	9

Instruções Gerais

1. Para obter o código inicial para a terceira fase, **Análise Semântica**. Se não fez a configuração do repositório oficial do professor como `upstream` na primeira parte, faça a configuração. Esse configuração deve ser feita apenas uma vez, conforme o @lst:comandos:iniciais.

```
# Conectar o repositório oficial do professor como 'upstream'.
git remote add upstream https://github.com/rogerioag/tppcompiler-code-start.git

# Obter os arquivos para a fase 3, Análise Semântica.
git fetch upstream
git merge upstream/main -m "Merge upstream Fase 3 - Semântico"
git push origin main
```

2. Siga a estrutura fornecida para desenvolver o trabalho, pois será a mesma

estrutura que deverá ser entregue ao final do projeto. Na última fase iremos adicionar outras partes do projeto nesse repositório.

Se a obtenção do código da nova fase funcionou, o diretório: `src/tppsema/` foi adicionado ao repositório do seu projeto, é nesse diretório que tem os arquivos que você irá trabalhar na análise semântica. Ao terminar a implementação envie para o seu `github`, conforme o @lst:comandos:envio.

```
git add .
git commit -m "feat: Implementação da Análise Semântica"
git push origin main
```

3. Após o envio para o seu `github`, acesse o link https://github.com/ e verifique o resultado da execução dos testes. Se algum teste falhar, corrija o problema e repita o processo. Os testes podem ser executados localmente com o comando: `pytest -v tests/tppsema test.py`.
4. Prepare o relatório com a descrição do trabalho e da implementação e exemplos de entrada e saída gerada pelo Analisador Semântico.
5. Deverá ser entregue o repositório compactado via moodle, conforme descrito na [Seção @sec:tpp:doc:entrega]. Com o código fonte da implementação completa e o relatório em **pdf** no formato de artigo conforme o modelo descrito na [Seção @sec:tpp:doc:entrega].
6. O projeto deve seguir a estrutura de diretórios e arquivos, disponível no repositório.
7. O relatório dessa segunda fase, deve conter o conteúdo das partes, Análise Léxica e Sintática, e conteúdo relacionado a esta terceira fase, Análise Semântica, incluindo exemplos de entrada e saída gerada pelo Analisador Semântico.
8. Deverão ser entregues:
   a) O código fonte da implementação.
   b) Relatório em **pdf** que pode ser feito no formato do `LibreOffice` ou no `Latex`.
9. O projeto deve seguir a estrutura de diretórios e arquivos, disponível no formato. A estrutura do projeto é apresentada na Figura \ref{fig:formato:entrega}.

Implementação

Com as Análises Léxica e Sintática já concluídas, iremos realizar a verificação das **Regras Semânticas** e espera-se ter como saída uma Árvore Sintática Abstrata (AST) podada.

A implementação da *Análise Semântica* deverá ser realizando utilizando o mesmo conjunto de ferramentas compatíveis utilizadas na Análise Léxica e Análise Sintática. O código base para a implementação está disponível no link Código de Start do Moodle.

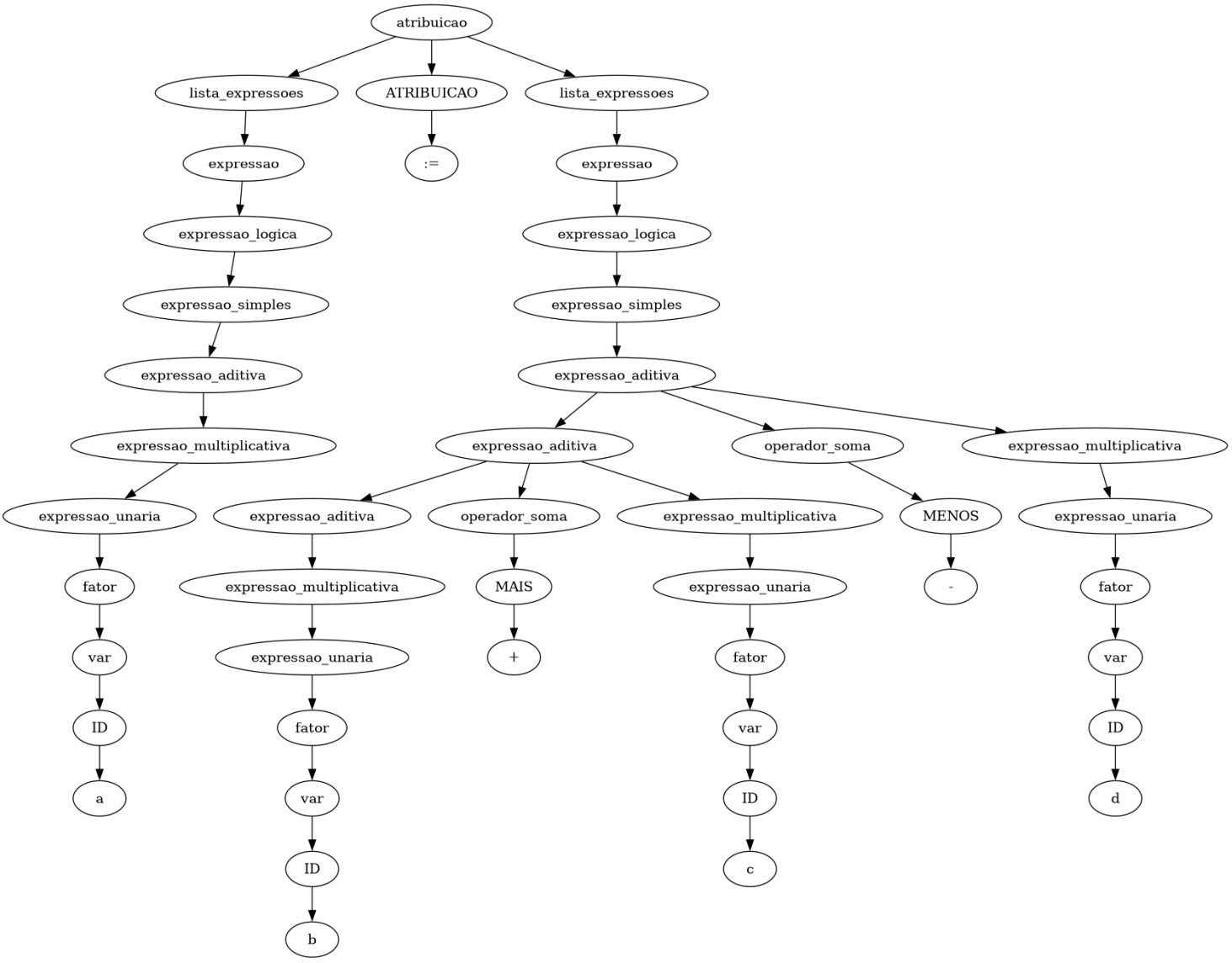
Para a terceira parte do trabalho, a **AST** deve ser percorrida utilizando as informações da *Tabela de Símbolos* iniciada na Análise Sintática. A análise semântica consiste da *"segunda passada"* do compilador no código, pois o código fonte é novamente analisado desde o início na **AST**, porém agora para realizar a sua anotação de atributos. A tabela de símbolos deve ser gerada/atualizada por meio das estratégias apresentadas em sala de aula.

O principal motivo da **Análise Semântica** é verificar se existem erros de contexto para a linguagem **TPP** apresentada na primeira e segunda parte deste trabalho.

Árvore Sintática Abstrata

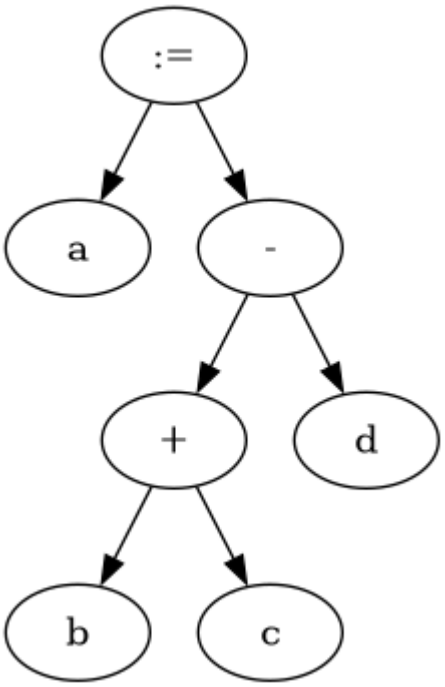
Após a análise semântica e geração de dados referentes a esta passagem, é esperado como saída uma árvore sintática abstrata anotada (ASTO).

Tomemos como exemplo uma atribuição `a := b + c - d`, de acordo com a sintaxe da linguagem **TPP** (gramática v1.1) temos a subárvore representada na Figura \ref{fig:atribuicao}. O lado esquerdo também é uma `lista_expressoes`, e a soma e a subtração estão no mesmo nível (`expressao_aditiva`) e associam à esquerda, ou seja, a expressão é calculada como `(b + c) - d`.



Após a verificação das regras semânticas, pode-se fazer uma poda dos nós internos da árvore para facilitar a geração de código. A Figura \ref{fig:atribuicao:poda} apresenta a **Árvore Sintática Abstrata Simplificada/Podada**.

A simplificação da árvore irá facilitar a fase de Geração de Código. No caso da *atribuição* o filho à esquerda `a` é a variável que irá receber o *store*, a subárvore do lado direito terá a subárvore com expressão. Descendo mais à esquerda, carrega-se o valor de `b`, visitando o nó à direita carrega-se o valor de `c` e então ao retornar para o pai `(+)` calcula-se a adição de `b` e `c`. Com a subárvore à esquerda de `(-)` resolvida, visita-se o lado direito e carrega-se o valor de `d`, voltando-se ao nó `(-)` a subtração é calculada com o resultado da expressão `b + c` e o valor de `d`.



O Código que será gerado na próxima fase para uma `atribuicao` é apresentado no Código:

```
%2 = load i32, i32* @b, align 4
%3 = load i32, i32* @c, align 4
%4 = add nsw i32 %2, %3
%5 = load i32, i32* @d, align 4
%6 = sub nsw i32 %4, %5
store i32 %6, i32* @a, align 4
```

Árvore Sintática das extensões da TPP v1.1

As construções da gramática v1.1 produzem os nós abaixo. Os nós `bloco` e `NEWLINE` não aparecem na árvore: um `bloco` é sempre representado por um nó `corpo`.

Construção	Nó	Filhos
`a, b := c, d` (dentro de funções)	`atribuicao`	`lista_expressoes`, `ATRIBUICAO`, `lista_expressoes`
`a := b := 0` (dentro de funções)	`atribuicao`	`lista_expressoes`, `ATRIBUICAO`, `atribuicao` (aninha à direita)

Construção	Nó	Filhos
`a, b := 1, 2` (escopo global)	`inicializacao_variaveis`	`lista_alvos`, `ATRIBUICAO`, `lista_expressoes`; com parênteses, `ABRE_PARENTESE` e `FECHA_PARENTESE` em volta de `lista_alvos`
`inteiro: v[5] := {1, 2}`	`declaracao_variaveis`	`tipo`, `DOIS_PONTOS`, `lista_variaveis`, `ATRIBUICAO`, `lista_expressoes`
`inteiro: a := b := 1`	`declaracao_variaveis`	`tipo`, `DOIS_PONTOS`, `lista_variaveis`, `ATRIBUICAO`, `atribuicao_encadeada`
elo de `a := b := 1`	`atribuicao_encadeada`	`lista_alvos` (com uma única `var`), `ATRIBUICAO`, `lista_expressoes` ou outra `atribuicao_encadeada`
`(a, b)`	`tupla`	`ABRE_PARENTESE`, `lista_expressoes`, `VIRGULA`, `expressao`, `FECHA_PARENTESE`
`{1, 2, 3}`	`inicializacao_arranjo`	`ABRE_CHAVE`, `lista_expressoes`, `FECHA_CHAVE`
`(inteiro, flutuante) f(...)`	`tipos_retorno`	`ABRE_PARENTESE`, `lista_tipos`, `FECHA_PARENTESE`
`retorna(a, b)`	`retorna`	`RETORNA`, `ABRE_PARENTESE`, `lista_expressoes`, `FECHA_PARENTESE` (também com um único valor)

Pontos de atenção ao percorrer a árvore:

- Na `tupla`, o último elemento fica fora da lista: `(a, b, c)` tem `lista_expressoes` com `a, b` e uma `expressao` com `c`. Para obter os elementos, junte os dois.
- Formas com o mesmo significado podem gerar árvores diferentes: dentro de uma função, `(a, b) := 1, 2` tem um único alvo (`tupla`), enquanto no escopo global tem dois alvos (`lista_alvos`). Normalize conforme o item 5.2 antes de comparar alvos e valores.
- Em `atribuicao_encadeada`, percorra os nós aninhados até chegar à `lista_expressoes` do valor.

Testes

Alguns casos de testes estão disponíveis no moodle institucional junto com essa especificação no código inicial fornecido. O `Github Classroom` executará os testes automaticamente. Serão executados esses testes e outros testes que o professor julgar necessário durante a avaliação desta parte do trabalho.

- **Sema:** programa de exemplo `TPP` de **entrada** na linha de comando.
- **Saída:** Árvore Podada ou uma lista de erros que foram identificados.

No Código Start da Semântica no Github Classroom tem um diretório `tests` com os testes para a *Análise Semântica*.

É necessário desenvolver a parte semântica e fazer o tratamento de erros. Está sendo disponibilizado uma classe de tratamento de erros que será utilizada em todas as fases do projeto, a `myerror.py` que carrega uma lista de erros composta por pares (chave:valor) formada por *códigos de erros* e as *mensagens de erros* correspondentes.

Nas fases anteriores provavelmente você alterou os arquivos relacionados à Análise Léxica e Sintática, copie seus arquivos para atualizar as versões iniciais que estão nesse *Código de Start* para a parte Semântica.

Código inicial da Análise Semântica

O código inicial segue a mesma organização das fases anteriores (`src/tpplexer/` e `src/tppparser/`):

```
src/tppsema/
├── __init__.py                    # Factory get_sema e exportações
├── base.py                        # Classe base abstrata (BaseSema)
├── SemaErrorMessages.properties   # Mensagens de erro e aviso da semântica
└── sema/                          # Estratégia padrão ("sema")
    ├── tppsema.py                 # TPPSema: aplica as regras e devolve a árvore
podada
    ├── symbol_table.py            # Tabela de símbolos (a implementar)
    ├── pruning.py                 # Poda da árvore sintática (a implementar)
    └── rules/                     # Uma regra por categoria desta especificação
(a implementar)
        ├── functions.py           # 1. Funções e Procedimentos
        ├── variables.py           # 2. Variáveis
        ├── coercions.py           # 3. Atribuição de tipos distintos e coerções
implícitas
        ├── arrays.py              # 4. Arranjos
        ├── assignments.py         # 5. Atribuição múltipla, encadeada e tuplas
        ├── returns.py             # 6. Retorno múltiplo
        └── expressions.py         # 7. Expressões
```

A Análise Semântica é executada pelo `src/tpp_compiler.py` com a opção `--sema`, que executa antes as análises léxica e sintática. A semântica só é executada quando a árvore sintática não tem erros. Cada módulo de `rules/` tem uma função `verificar(arvore, tabela, sema)`, chamada pelo `TPPSema`; os erros e avisos devem ser impressos por `sema.emitir(chave, *args, linha=..., coluna=...)`, que trata a opção `-k`.

Para que os testes possam ser automatizados, o `Github` executa o `tests/tppsema_test.py` a cada `commit`. Ele executa o `src/tpp_compiler.py --sema sema -k` com cada arquivo `.tpp` de `tests/sem-tests/` e compara a saída produzida com a saída esperada, no arquivo `.tpp.sem.out` correspondente (como os `.lex.out` e `.syn.out` das fases anteriores):

```
$ pytest tests/tppsema_test.py -v
```

A ideia é que em uma *execução normal* as mensagens de erros sejam recuperadas e impressas na saída padrão (terminal).

Exemplo de saída para teste com erro em *execução normal*:

[columns]

[column=0.5]

Arquivo: `tests/sem-tests/sem-test-001.tpp`

```
{*Erro: Função 'principal' não declarada*}
{*Aviso: Variável 'a' declarada e não utilizada*}
{*Aviso: Variável 'b' declarada e não utilizada*}

inteiro: a
flutuante: b
```

Execução:

```
$ python src/tpp_compiler.py --sema sema tests/sem-tests/sem-test-001.tpp
```

[column=0.5]

Saída gerada:

```
Erro[03][01]: Função 'principal' não declarada.
Aviso[01][10]: Variável 'a' declarada e não utilizada.
Aviso[02][11]: Variável 'b' declarada e não utilizada.
```

[/columns]

Já em uma *execução de testes automatizados* seja fornecido um parâmetro `-k` que imprima o código de erro apenas, possibilitando que o código apareça em uma lista semelhante à de *tokens* gerada para a primeira fase.

Exemplo de saída para teste com erros em *execução de testes automatizados*:

[columns]

[column=0.5]

Arquivo: `tests/sem-tests/sem-test-001.tpp`

```
{*Erro: Função 'principal' não declarada*}
{*Aviso: Variável 'a' declarada e não utilizada*}
{*Aviso: Variável 'b' declarada e não utilizada*}

inteiro: a
flutuante: b
```

Execução:

```
$ python src/tpp_compiler.py --sema sema -k tests/sem-tests/sem-test-001.tpp
```

[column=0.5]

Saída gerada:

```
ERR-SEM-MAIN-NOT-DECL
WAR-SEM-VAR-DECL-NOT-USED
WAR-SEM-VAR-DECL-NOT-USED
```

[/columns]

O arquivo `src/tppsema/SemaErrorMessages.properties` contém a seção de erros e avisos da *Semântica*:

```
[SemaErrors]
ERR-SEM-MAIN-NOT-DECL=Função 'principal' não declarada.
ERR-SEM-VAR-NOT-DECL=Variável '{}' não declarada.
WAR-SEM-VAR-DECL-NOT-INIT=Variável '{}' declarada e não inicializada.
WAR-SEM-VAR-DECL-NOT-USED=Variável '{}' declarada e não utilizada.
ERR-SEM-ARRAY-INDEX-NOT-INT=Índice de array '{}' não 'inteiro'.
ERR-SEM-ARRAY-INDEX-OUT-OF-RANGE=Índice de array 'A' fora do intervalo (out of
range).
ERR-SEM-FUNC-RET-TYPE-ERROR=Função '{}' deveria retornar '{}', mas retorna '{}'.
ERR-SEM-CALL-FUNC-NOT-DECL=Chamada à função '{}' que não foi declarada.
ERR-SEM-CALL-FUNC-WITH-FEW-ARGS=Chamada à função '{}' com número de parâmetros
menor que o declarado.
ERR-SEM-CALL-FUNC-WITH-MANY-ARGS=Chamada à função '{}' com número de parâmetros
maior que o declarado.
ERR-SEM-CALL-FUNC-MAIN-NOT-ALLOWED=Chamada à função 'principal' não permitida.
```

```
WAR-SEM-VAR-DECL-PREV=Variável '{}' declarada anteriormente com o tipo '{}'.
WAR-SEM-VAR-DECL-INIT-NOT-USED=Variável '{}' declarada e inicializada, mas não
utilizada.
WAR-SEM-ATR-DIFF-TYPES-IMP-COERC-OF-VAR=Atribuição de tipos distintos. Coerção
implícita do valor de '{}' do tipo '{}' para '{}' que é '{}'.
WAR-SEM-ATR-DIFF-TYPES-IMP-COERC-OF-EXP=Atribuição de tipos distintos. Coerção
implícita do valor de '{}' do tipo '{}' para '{}' que é '{}'.
WAR-SEM-ATR-DIFF-TYPES-IMP-COERC-OF-RET-VAL=Atribuição de tipos distintos. Coerção
implícita do valor retornado por '{}' do tipo '{}' para '{}' que é '{}'.
WAR-SEM-ATR-DIFF-TYPES-IMP-COERC-OF-NUM=Atribuição de tipos distintos. Coerção
implícita do valor atribuído '{}' do tipo '{}' para '{}' que é '{}'.
WAR-SEM-ATR-DIFF-TYPES-IMP-COERC-OF-FUNC-ARG=Chamada à função '{}' com Coerção
implícita do valor do argumento tipo '{}' diferente do parâmetro declarado '{}'.
WAR-SEM-FUNC-DECL-NOT-USED=Função '{}' declarada, mas não utilizada.
WAR-SEM-CALL-REC-FUNC-MAIN=Chamada recursiva para 'principal'.
ERR-SEM-INESPERADO=Erro inesperado.
WAR-SEM-ARRAY-INIT-EXCESS-ELEMS=Elementos excedentes no inicializador de '{}' foram
descartados.
ERR-SEM-ARRAY-INIT-DIM=Inicializador de '{}' tem mais dimensões que o arranjo.
ERR-SEM-ARRAY-INIT-IN-EXP=Inicializador de arranjo não pode ser usado em uma
expressão.
ERR-SEM-ATR-INVALID-TARGET=Alvo inválido na atribuição: '{}' não é uma variável.
ERR-SEM-ATR-DIFF-NUM-TARGETS-VALUES=Atribuição com {} alvos e {} valores.
ERR-SEM-TUPLE-NOT-ALLOWED=Tupla não permitida neste contexto.
ERR-SEM-FUNC-RET-NUM-ERROR=Função '{}' deveria retornar {} valores, mas retorna {}.
ERR-SEM-CALL-FUNC-MULTI-RET-IN-EXP=Chamada à função '{}', que retorna {} valores,
usada em uma expressão.
WAR-SEM-EXP-STMT-NO-EFFECT=Expressão usada como comando não tem efeito.
WAR-SEM-REL-CHAIN=Comparação encadeada: o resultado de uma comparação é comparado
com outro valor.
```

Para a regra `**Erro:** *Função principal não declarada.*` foi criada uma chave `ERR-SEM-MAIN-NOT-DECL` com a mensagem no arquivo `SemaErrorMessages.properties`. Da mesma maneira que nas fases anteriores se passado um parâmetro `-k` os erros apresentados na saída padrão, o terminal, deve apenas imprimir a chave para efeitos de comparação com a saída esperada.

Na impressão feita pela classe `MyError`, que foi definida no arquivo `myerror.py`, a distinção entre erros e avisos deve ser feita.

Documentação

Durante toda a disciplina o aluno criará uma documentação formal da implementação do compilador para a linguagem. Sendo os relatório com conteúdo acumulativo, isto é, as fases subsequentes irão complementar o conteúdo existente das fases anteriores. Neste ponto do trabalho o aluno deverá incluir na documentação:

- Estratégias utilizadas para a realização da Análise Semântica.
- Estratégia utilizada para a geração da Tabela de Símbolos.
- Qualquer decisão de projeto referente à semântica da linguagem TPP.

O relatório deve ser escrito no formato de artigo[1] disponível no link, faça uma cópia para seu `Overleaf`. O número mínimo em cada etapa é de 12 páginas.

Avaliação

Será avaliado se a análise semântica executa e se todos os erros e *warnings* semânticos são mostrados corretamente para os exemplos de testes.

Serão avaliados, dentre outros critérios:

a) Da implementação:

- O funcionamento do programa.
- O capricho e a organização na elaboração do projeto.
- A corretude da implementação em relação ao que foi pedido no trabalho.
- A colocação em prática dos conceitos que foram discutidos em sala de aula de forma correta.
- A qualidade do projeto e da implementação (descrição e elaboração do projeto e o passo a passo da implementação).

b) Do relatório:

- O conteúdo e a forma que foi apresentado, se o formato é o mesmo solicitado.
- Organização das ideias e do processo de tradução.
- O capricho na elaboração e na formatação do texto, bem como o conteúdo do texto.
- Não serão avaliados os trabalhos:
  a) Que chegarem fora do prazo.
  b) Que não forem feitos nas ferramentas solicitadas.
  c) Que não estão no formato especificado.
  d) Que não foram compactados em um só arquivo.
  e) Que não tiverem identificação (nome e matrícula).
  f) Que forem cópias de outros trabalhos ou materiais da internet.
  g) Que não seguirem todas estas instruções.
- Não se esqueça que o trabalho vale **10,0** e contribui para o cálculo da nota final.

Entrega e apresentação

O trabalho será **individual** e deverá ser entregue até o dia **19/11/2026** no moodle da disciplina em um pacote compactado e apresentado no mesmo dia. A estrutura do projeto com os arquivos do projeto (fonte e relatório) deve ser compactada (zipados) e o arquivo compactado deve ser enviado pelo moodle utilizando a opção de submissão "Trabalho 3a. parte - Análise Semântica", o nome do arquivo compactado deve seguir o padrão de nomes do formato.

Deverá ser especificado na entrega o mecanismo de execução da varredura para a realização da correção.

Obs.: Favor utilizar ZIP como forma de compactação.

Leitura Recomendada {.allowframebreaks}

Important

Leitura Recomendada

Capítulo 6: Análise Semântica, Seção 6.3. A Tabela de Símbolos.

Livro: LOUDEN, Kenneth C. Compiladores: princípios e práticas. São Paulo, SP: Thomson, c2004. xiv, 569 p. ISBN 8522104220.

Important

Leitura Recomendada

Capítulo 2: Um tradutor simples dirigido por Sintaxe. Seção 2.7 -- Tabela de Símbolos

Livro: AHO, Alfred V. et al. Compiladores: princípios, técnicas e ferramentas. 2. ed. São Paulo, SP: Pearson Addison-Wesley, 2008. x, 634 p. ISBN 9788588639249.

Important

Leitura Recomendada

Capítulo 4: Análise Sensível ao Contexto

Livro: COOPER, Keith D.; TORCZON, Linda. Construindo compiladores. Rio de Janeiro, RJ: Elsevier/Campus, 2014. xxiv, 656 p. ISBN 9788535255645. Disponível em: https://research.ebsco.com/linkprocessor/plink?id=debd0a1f-ba89-3ee8-b0a1-2c929486cc51. Acesso em: 12 mar. 2024.

Word Cloud



Referências {.fragile .allowframebreaks}

---

1. Formato para o relatório: https://www.overleaf.com/read/fcqmjkfgktgt ↩